Shamba Direct Marketplace

Architecture, Database, Subscription and Payment Plan

Connecting Farmers. Empowering Millers. Growing Kenya.

First deliverables: architecture, database schema, homepage design, subscription workflow, payment flow. The brand name is stored in one config value so it can be changed later.

1. Architecture

Stack (hosted on Netlify)

- **Frontend and app:** Next.js (React) with Tailwind CSS, mobile-first.
- **Backend:** Next.js route handlers / Netlify Functions (Node.js). Payment callbacks and scheduled jobs run here.
- **Database:** PostgreSQL on a managed host (Neon or Supabase). Netlify does not host databases, so this is a separate service.
- **File storage:** Supabase Storage or Cloudflare R2 (private bucket for verification documents).
- **Auth:** email or phone plus password (Argon2 or bcrypt), session cookies, roles: farmer, miller, admin.
- **Email and SMS:** Resend or Brevo for email; Africa's Talking for SMS.
- **Scheduled jobs:** Netlify Scheduled Functions for subscription expiry checks and reminders.

Layers

1. Browser: phone-first pages, compressed image uploads, low-bandwidth.
2. Next.js app with role-based route protection.
3. API layer: validation (Zod), rate limiting, server-side authorization on every request.
4. PostgreSQL plus object storage.
5. External services: payment gateway, email/SMS.

Security basics: server-side role checks, input validation, rate limiting on login and payment endpoints, no card or PIN storage, private verification documents, admin audit log, daily database backups.

Future Android app: keep all logic behind a versioned REST API (/api/v1/...) so a mobile app can reuse it.

2. Database Schema (PostgreSQL)

Table	Main fields
users	id, email, phone, password_hash, role (farmer/miller/admin), status (pending/active/suspended/rejected), created_at
profiles	id, user_id, account_type, business_name, registration_number, county, town, address, contact_name, whatsapp_number, verified, verified_at
verification_documents	id, profile_id, doc_type, file_path (private), status, reviewed_by, reviewed_at, notes
crops	id, name (maize, wheat, rice, beans, green grams, millet, sorghum, other)
listings	id, profile_id, crop_id, quantity_kg, price, price_unit, moisture_pct, grade, harvest_date, certifications, county, town, transport, availability_date, status (available/negotiation/reserved/sold), review_status, created_at
listing_photos	id, listing_id, file_path, sort_order
requirements	id, profile_id, crop_id, quantity_kg, max_moisture_pct, min_grade, packaging, delivery_county, delivery_location, needed_by, indicative_price, price_unit, invite_quotes, status, created_at

Table	Main fields
matches	id, listing_id, requirement_id, score, notified_at
plans	id, audience, name, interval (monthly/quarterly/annual), price_kes, features (json), active. Admin-editable.
subscriptions	id, user_id, plan_id, status (pending/active/expired/cancelled), starts_at, ends_at, auto_renew
payments	id, user_id, subscription_id, provider, amount_kes, phone, provider_request_id, provider_receipt (unique), status (pending/success/failed/cancelled), raw_callback (json), created_at, confirmed_at
conversations, messages	conversation (listing or requirement, two participants); messages: sender_id, body, attachment, sent_at, read_at
offers	id, conversation_id, from_user, price, quantity_kg, delivery_date, payment_terms, status (open/accepted/rejected/countered)
orders	id, offer_id, farmer_id, miller_id, agreed_price, quantity_kg, delivery_date, payment_terms, status (agreed/in_transit/delivered/completed/disputed), agreement_pdf_path
reviews	id, order_id, reviewer_id, reviewee_id, rating, comment
reports, disputes	id, reporter_id, target user or listing, reason, status, admin_notes, resolved_at
notifications	id, user_id, type, message, channel (in-app/email/sms), sent_at
audit_logs	id, admin_id, action, entity, entity_id, details (json), created_at
settings	key, value (brand name, tagline, contact details, free-tier limits)

3. Access Rules

Feature	Free (registered)	Active subscriber
Browse limited listings	Yes	Yes
See contact details and message	No	Yes
Post listing or requirement	1 draft	Per plan limit
Matching alerts	No	Yes
Offers and orders	No	Yes

Every premium endpoint checks that the subscription status is active and ends_at is in the future, on the server.

4. Subscription Workflow

1. User registers as farmer or miller and verifies phone or email.
2. Profile is completed; admin reviews and verifies (badge awarded).
3. User opens Subscription Plans, sees benefits for their role, selects plan and interval.
4. Checkout creates a pending subscription and a pending payment.
5. Payment flow runs (section 5).
6. On confirmed success, subscription becomes active; ends_at = now plus plan interval. Early renewal extends from the current ends_at.
7. Reminders at 7, 3 and 1 days before expiry (email or SMS).
8. A daily scheduled job marks overdue subscriptions expired; premium features lock, data is kept.
9. Receipts and history are in the dashboard; admins see revenue, renewals and expiries.

5. Payment Flow (M-Pesa)

Provider choice

- **Daraja (Safaricom direct):** lowest fees, but needs a Paybill/Till, go-live approval and more setup.
- **IntaSend or Pesapal:** faster to start, add card payments, handle reconciliation. Higher fees.
- **Recommendation for the pilot:** start with IntaSend or Pesapal; move to Daraja when volume justifies it. Keep payment code behind one provider interface so switching is easy.

STK Push sequence

1. User picks a plan and enters their M-Pesa number.
2. Server validates the number, creates a pending payment, and calls the provider's STK Push API.
3. User gets the prompt on their phone and enters their PIN on the phone itself. The site never sees or stores the PIN.
4. The page shows "Waiting for confirmation" and polls the payment status endpoint.
5. The provider sends a callback (webhook). The server verifies it: signature or secret check, payment ID matches a pending record, amount equals the plan price, receipt not used before. Where supported, it also queries the provider's transaction-status API.
6. Only after verification does the server mark the payment successful and activate the subscription, in one database transaction.
7. Failed or cancelled payments show a clear message with a retry option.
8. A receipt is emailed and shown on the payment status page (successful, pending, failed).

Never activate on a screenshot, a manually typed code, or a client-side message. Keep a manual admin override for genuine reconciliation cases, logged in the audit log.

Edge cases: late callback (poll provider status after 2 minutes), duplicate callbacks (idempotent via unique receipt), wrong amount (flag for admin), user closes the page (webhook still activates).

6. Homepage Design

Look: green (growth), gold (harvest), white background; real agricultural photos. No fabricated users, transactions or testimonials.

1. Header: logo, Browse Produce, Buying Requirements, Plans, Login. Menu collapses on phones.
2. Hero: headline "SELL YOUR HARVEST DIRECTLY TO MILLERS. BUY GRAIN DIRECTLY FROM FARMERS." with supporting text and four large buttons: Join as a Farmer, Register as a Miller, Browse Available Produce, View Subscription Plans.
3. How it works: Register, Get verified, Subscribe, Connect and trade.
4. Available crops: maize, wheat, rice, beans, sorghum, green grams.
5. Recent listings and current buying requirements (live data; honest empty state until real data exists).
6. Subscription benefits and starting prices.
7. Verified participants (real accounts only), with a note that verification does not guarantee quality or performance.
8. FAQ.
9. Testimonials: hidden until genuine ones exist.
10. Footer: Terms, Privacy, Refund, Disputes and Safety, Contact.

Mobile rules: large tap targets, single-column layout, images compressed before upload, system fonts, minimal JavaScript on public pages.

7. Pages (20)

Home, About, How It Works, Browse Cereals, Find Millers, Buying Requirements, Farmer Registration, Miller Registration, Subscription Plans, Checkout, Login, Farmer Dashboard, Miller Dashboard, Admin Dashboard, Contact, FAQ, Terms, Privacy, Refund and Cancellation, Dispute Resolution and Safety.

Legal pages are drafts for a qualified Kenyan lawyer to review, including data protection obligations under Kenyan law. The platform must not promise payment, quality or delivery guarantees it does not provide.

8. Build Order

1. Project setup, database, auth, roles, registration and login.
2. Profiles and admin verification.
3. Plans, checkout and M-Pesa payment with webhook (sandbox testing).
4. Listings and buying requirements with search and filters.
5. Matching and notifications.
6. Messaging, offers and orders.
7. Admin dashboard, reports, audit log.
8. Public pages, legal drafts, testing, deployment on Netlify.

9. Success Test

A farmer in Kitale subscribes, lists maize and receives an inquiry from a Nairobi or Mombasa miller. A miller subscribes, posts a maize requirement, finds matching farmers and negotiates in-platform. Both flows are tested end to end in the payment sandbox before launch.